SQL Server

Part 1 (SQL Fundamentals & Performance)

Question 1: Explain Clustered Index and Non-Clustered Index

Interviewer

What is the difference between a Clustered Index and a Non-Clustered Index? Which one is faster?

This is one of the most frequently asked SQL interview questions.

Answer

An index improves the speed of data retrieval by reducing the amount of data scanned.

Clustered Index

A Clustered Index determines the **physical order** of rows in a table.

There can be **only one** clustered index because the table's rows can be stored in only one physical order.

Example

Employee Table

EmployeeId

1

2

3

4

5

Data is physically stored in this order.

Example

CREATE CLUSTERED INDEX IX_Employee_Id

ON Employees(EmployeeId);

Advantages

✓ Fast range searches

✓ Efficient sorting

✓ Good for primary keys

Non-Clustered Index

A Non-Clustered Index is a separate structure that stores the indexed key and a pointer to the actual row.

Example

Non-Clustered Index

Name

↓

John → Row 100

David → Row 200

Satheesh → Row 300

The table data remains stored separately.

Example

CREATE NONCLUSTERED INDEX IX_Employee_Name

ON Employees(Name);

Comparison

Feature	Clustered	Non-Clustered
Physical order	Yes	No
Number allowed	One	Many
Data stored	Table rows	Index with row locator
Best for	Primary key, range queries	Search columns, filtering

---

Interview Question

Which is faster?

Senior Answer

It depends on the query.

- Clustered indexes are efficient for range scans and ordered retrieval.
- Non-clustered indexes are efficient for selective lookups.
- SQL Server chooses the best index based on statistics and the execution plan.

Question 2

Explain Covering Index

Answer

A covering index contains **all columns required by a query**, allowing SQL Server to satisfy the query using only the index.

Example

SELECT Name, Salary

FROM Employees

WHERE Department = 'IT';

Create Index

CREATE NONCLUSTERED INDEX IX_Department

ON Employees(Department)

INCLUDE (Name, Salary);

Why is it faster?

Without Covering Index

Index Seek

↓

Key Lookup

↓

Table

Extra lookup is required.

With Covering Index

Index Seek

↓

Return Data

No table lookup.

Interview Question

When should you use a Covering Index?

For frequently executed queries where avoiding additional table lookups significantly improves performance.

Question 3

Explain Execution Plan

Very Important.

Answer

An Execution Plan shows **how SQL Server executes a query**.

It reveals:

- Table scans
- Index seeks
- Index scans
- Key lookups
- Joins
- Estimated cost

Example

SELECT *

FROM Employees

WHERE EmployeeId = 100;

If indexed

Index Seek

If not

Table Scan

Interview Question

Difference between Index Scan and Index Seek?

Index Seek	**Index Scan**
Directly locates matching rows	Reads many index entries
Faster	Slower for selective queries
Uses search predicates	Scans more data

Senior Tip

Always examine the **Actual Execution Plan** when tuning production queries.

Question 4

Explain SQL Query Optimization

Common Optimization Techniques

✓ Create appropriate indexes

✓ Avoid SELECT *

✓ Return only required columns

✓ Filter data early

✓ Use parameterized queries

✓ Review execution plans

✓ Avoid functions on indexed columns in predicates when possible

✓ Keep statistics updated

Bad Example

```
SELECT *
FROM Employees;
```

Better Example

```
SELECT EmployeeId,
    Name,
    Salary
FROM Employees;
```

Interview Question

Why avoid SELECT *?

Because it:

- Returns unnecessary data
- Uses more bandwidth
- Consumes more memory
- Can prevent covering index usage

Question 5

Explain Stored Procedure

Answer

A Stored Procedure is a precompiled SQL program stored in the database.

Example

```
CREATE PROCEDURE GetEmployee
   @Id INT
AS
BEGIN
   SELECT *
   FROM Employees
   WHERE EmployeeId = @Id;
END;
```

Execute

```
EXEC GetEmployee @Id = 10;
```

Advantages

✓ Reusable

✓ Easier maintenance

✓ Parameterized

✓ Supports permissions management

Interview Question

Stored Procedure vs Inline SQL?

Senior Answer

Stored Procedures centralize business logic in the database and can simplify maintenance, while parameterized SQL in application code often provides better flexibility. The choice depends on architecture, maintainability, and performance requirements.

Question 6

Explain Views

A View is a virtual table based on a SQL query.

Example

```
CREATE VIEW ActiveEmployees
AS
SELECT EmployeeId,
    Name
FROM Employees
WHERE IsActive = 1;
```

Query

```
SELECT *
FROM ActiveEmployees;
```

Advantages

✓ Simplifies complex queries

✓ Reusable

✓ Can restrict exposed columns

Question 7

Explain CTE (Common Table Expression)

A CTE creates a temporary named result set within a query.

Example

```
WITH EmployeeCTE AS
(
  SELECT EmployeeId,
      Salary
  FROM Employees
)
SELECT *
FROM EmployeeCTE;
```

Interview Question

Why use a CTE?

For:

- Recursive queries
- Improved readability
- Breaking complex logic into manageable parts

Question 8

Explain Temporary Table vs Table Variable

Temporary Table

```
CREATE TABLE #Employee
(
   Id INT,
   Name VARCHAR(100)
);
```

Table Variable

```
DECLARE @Employee TABLE
(
   Id INT,
   Name VARCHAR(100)
);
```

Comparison

Temporary Table	Table Variable
Better for large data	Better for small data sets
Supports indexes	More limited indexing options
Better optimizer statistics	Limited statistics (behavior varies by SQL Server version)

Interview Question

Which one performs better?

Senior Answer

Temporary tables generally perform better for larger datasets because they support richer statistics and indexing.

Question 9

Explain Functions

Scalar Function

Returns one value.

```
CREATE FUNCTION dbo.GetBonus
(
   @Salary DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
AS
BEGIN
   RETURN @Salary * 0.10;
END;
```

Table-Valued Function

Returns a table.

```
CREATE FUNCTION dbo.GetEmployees()
RETURNS TABLE
AS
RETURN
(
   SELECT *
   FROM Employees
);
```

Interview Question

Stored Procedure vs Function

Stored Procedure	Function
Can perform data modification	More restricted
Can return multiple result sets	Returns a value or table
Can be executed directly	Can be used within SQL expressions (depending on function type)

Question 10

Explain Window Functions

Very Frequently Asked

ROW_NUMBER()

```
SELECT Name,
    Salary,
    ROW_NUMBER() OVER(ORDER BY Salary DESC) AS RowNum
FROM Employees;
```

RANK()

If two salaries are equal:

1

2

2

4

DENSE_RANK()

1

2

2

3

Interview Question

Difference between RANK() and DENSE_RANK()

Senior Answer

RANK() leaves gaps after ties.

DENSE_RANK() does not.

⭐ **Scenario-Based Question**

Interviewer

A query that usually runs in **2 seconds** is now taking **30 seconds**.

How would you troubleshoot?

Strong Senior Answer

I would:

1. Review the Actual Execution Plan.
2. Check whether indexes are being used.
3. Identify table scans or expensive key lookups.
4. Review statistics and update them if necessary.
5. Check for blocking or deadlocks.
6. Verify parameter values (parameter sniffing may be a factor).
7. Check server CPU, memory, and disk I/O.
8. Compare the execution plan with a known good execution.

Top 10 Rapid-Fire SQL Questions

Q1. Clustered vs Non-Clustered Index?

Clustered determines physical row order; Non-clustered stores keys with row locators.

Q2. What is a Covering Index?

An index containing all columns needed by a query.

Q3. What is an Execution Plan?

A graphical or textual representation of how SQL Server executes a query.

Q4. Why avoid SELECT *?

It returns unnecessary data and can reduce performance.

Q5. What is a Stored Procedure?

A reusable SQL program stored in the database.

Q6. What is a View?

A virtual table based on a query.

Q7. What is a CTE?

A temporary named result set within a query.

Q8. Temp Table vs Table Variable?

Temporary tables are generally better for larger datasets.

Q9. What is ROW_NUMBER()?

A window function that assigns a unique sequential number to each row.

Q10. Why are indexes important?

They reduce the amount of data scanned and improve query performance.

Module 7 Progress

We've covered:

- Clustered Index

- Non-Clustered Index
- Covering Index
- Execution Plans
- Query Optimization
- Stored Procedures
- Views
- CTEs
- Temporary Tables
- Table Variables
- Functions
- Window Functions
- Performance Troubleshooting

Next (Module 7 – Part 2)

We'll cover advanced SQL topics that are almost guaranteed in senior interviews:

1. Transactions
2. ACID Properties
3. Isolation Levels
4. Deadlocks
5. Locking & Blocking
6. Parameter Sniffing
7. Partitioning
8. Query Hints
9. Index Maintenance
10. SQL Server vs PostgreSQL
11. PostgreSQL Index Types
12. JSON Support
13. Real Production Scenarios
14. Top 50 Advanced SQL Interview Questions

These topics are especially valuable because senior interviewers often use them to assess troubleshooting skills in production environments.

Excellent. This is the **most important SQL module** for **Senior .NET Developers (8–15 years)**.

From my experience interviewing senior candidates, **80% of SQL interview questions** are based on:

- Transactions
- Isolation Levels
- Deadlocks
- Locking
- Blocking
- Performance Tuning
- Production Issues

These topics separate **Junior/Mid-Level** developers from **Senior Engineers**.

Module 7 – SQL Server & PostgreSQL

Part 2 (Advanced SQL & Production Troubleshooting)

Question 11: Explain Transactions

Interviewer

What is a transaction? Why is it important?

Answer

A **transaction** is a sequence of one or more SQL operations treated as a single logical unit of work.

A transaction follows the principle:

All Success

OR

All Rollback

Example

Bank Transfer

Account A

↓

Debit ₹1000

↓

Account B

↓

Credit ₹1000

If credit fails

↓

Debit must also rollback.

SQL Example

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 1000
WHERE AccountId = 1;

UPDATE Accounts
SET Balance = Balance + 1000
WHERE AccountId = 2;

COMMIT;
Rollback
ROLLBACK;
```

Interview Question

When should you rollback?

Strong Answer

Rollback whenever a business operation cannot be completed successfully, ensuring the database remains consistent.

Question 12

Explain ACID Properties

Very Frequently Asked

ACID

Property	Meaning
A	Atomicity
C	Consistency
I	Isolation
D	Durability

Atomicity

Everything succeeds

or

Everything rolls back.

Consistency

Database remains valid before and after transaction.

Example

Bank Balance

Cannot become

-₹5000

unless business rules allow it.

Isolation

Transactions should not interfere with each other.

Durability

After COMMIT,

Data survives failures.

Interview Question

Which ACID property prevents partial updates?

Answer

Atomicity.

Question 13

Explain Isolation Levels

One of the Top SQL Questions.

SQL Server Isolation Levels

1. Read Uncommitted
2. Read Committed (default in SQL Server)
3. Repeatable Read
4. Serializable
5. Snapshot (optional, database setting)

Read Uncommitted

Allows

Dirty Reads.

Fastest

Least Safe.

Read Committed

Prevents Dirty Reads.

Default.

Repeatable Read

Prevents

Dirty Reads

-

Non-repeatable Reads.

Serializable

Highest Isolation.

Prevents

Dirty Reads

Non-repeatable Reads

Phantom Reads.

Snapshot

Uses row versioning.

Reduces blocking while providing a transactionally consistent view of data.

Comparison

Isolation Level	Dirty Read	Non-repeatable Read	Phantom Read
Read Uncommitted	Yes	Yes	Yes
Read Committed	No	Yes	Yes
Repeatable Read	No	No	Yes
Serializable	No	No	No
Snapshot	No	No	No (uses versioning for consistent reads)

Interview Question

Which isolation level is used most?

Senior Answer

READ COMMITTED is the default and commonly used because it provides a good balance between consistency and concurrency.

Question 14

Explain Deadlocks

Very Frequently Asked

Example

Transaction A

Locks

Table A

↓

Needs

Table B

Transaction B

Locks

Table B

↓

Needs

Table A

Both wait forever.

SQL Example

Session 1

BEGIN TRAN;

UPDATE Employee

SET Salary = Salary + 100

WHERE Id = 1;

Session 2

BEGIN TRAN;

UPDATE Department

SET Name='IT'

WHERE Id=1;

If each session later requests the other's locked resource, a deadlock can occur.

Interview Question

How do you prevent deadlocks?

Strong Answer

- Access tables in a consistent order.
- Keep transactions short.
- Commit quickly.
- Create appropriate indexes.
- Avoid unnecessary locking.
- Implement retry logic because SQL Server automatically selects a deadlock victim.

Question 15

Explain Locking and Blocking

Locking

SQL Server locks data to maintain consistency.

Types

- Shared Lock (S)
- Exclusive Lock (X)
- Update Lock (U)
- Intent Locks (IS, IX, etc.)

Blocking

Blocking occurs when one transaction waits for another transaction to release a lock.

Example

Transaction A

Updates

Employee

↓

Transaction B

Waits

Interview Question

Blocking vs Deadlock?

Blocking	**Deadlock**
Waiting	Circular waiting
Eventually completes	One transaction is terminated by SQL Server

---

Question 16

Explain Parameter Sniffing

Senior-level question.

Answer

SQL Server creates an execution plan based on the parameter values used during compilation.

If later executions have very different parameter values, the cached plan may not be optimal.

Example

First

EXEC GetEmployees 'IT'

10 rows.

Plan Cached.

Later

EXEC GetEmployees 'Sales'

1,000,000 rows.

Same plan reused.

Poor performance.

Interview Question

How do you fix Parameter Sniffing?

Strong Answer

Possible approaches include:

- OPTION (RECOMPILE) for specific queries
- Optimizing indexes and statistics
- Using local variables in specific scenarios
- Query Store (SQL Server)
- Rewriting the query when appropriate

Choose the solution based on the workload rather than applying one technique universally.

Question 17

Explain Table Partitioning

Large Table

Orders

500 Million Rows

Partition

2022

2023

2024

2025

Advantages

✓ Faster maintenance

✓ Improved manageability

✓ Efficient archival

Interview Question

Should every table be partitioned?

Answer

No.

Partitioning is beneficial for very large tables and specific workloads. It also introduces management complexity.

Question 18

Explain SQL Server vs PostgreSQL

Very Frequently Asked

SQL Server	**PostgreSQL**
Microsoft	Open Source
T-SQL	PL/pgSQL
Strong Microsoft ecosystem integration	Strong standards compliance
Enterprise features	Rich extensibility

PostgreSQL Advantages

✓ JSONB

✓ Advanced indexing

✓ Extensions

✓ Strong SQL standards support

Interview Question

Which database should you choose?

Senior Answer

It depends on business requirements, existing ecosystem, licensing, operational expertise, scalability needs, and application architecture.

Question 19

Explain JSON Support

SQL Server

SELECT *

FROM Orders

FOR JSON PATH;

PostgreSQL

SELECT data->>'Name'

FROM Employee;

using JSON/JSONB operators.

Interview Question

Which database has stronger native JSON capabilities?

Senior Answer

PostgreSQL's **JSONB** provides powerful indexing and querying capabilities for JSON documents.

Question 20

SQL Performance Best Practices

Always

✓ Use Indexes

✓ Avoid SELECT *

✓ Return only required columns

✓ Use Pagination

✓ Parameterize queries

✓ Review Execution Plans

✓ Keep Statistics Updated

✓ Keep Transactions Short

✓ Avoid RBAR ("Row By Agonizing Row") when set-based operations are possible

✓ Use appropriate data types

⭐ **Production Scenario 1**

Interviewer

Production database CPU suddenly reaches 100%.

How do you investigate?

Strong Senior Answer

I would:

1. Identify top CPU-consuming queries using Query Store or DMVs.
2. Review execution plans.
3. Check for missing or unused indexes.
4. Look for parameter sniffing issues.
5. Verify blocking and deadlocks.
6. Check statistics and index fragmentation.
7. Review recent deployments and schema changes.
8. Monitor CPU, memory, and I/O metrics.

⭐ **Production Scenario 2**

Interviewer

Users report that saving records takes 30 seconds.

How do you troubleshoot?

Strong Answer

I would verify:

- Blocking sessions
- Long-running transactions
- Deadlocks
- Slow stored procedures

- Execution plans
- Database connection pool
- Network latency
- Application logs
- Recent deployments

⭐ **Production Scenario 3**

Interviewer

A query was fast yesterday but is slow today.

What could be the reasons?

Senior Answer

Possible causes:

- Parameter sniffing
- Statistics became outdated
- Data volume increased
- Missing or fragmented indexes
- Blocking
- Different execution plan
- Resource contention (CPU, memory, I/O)

Top 15 Rapid-Fire SQL Questions

Q1. What is ACID?

Atomicity, Consistency, Isolation, Durability.

Q2. Default Isolation Level?

READ COMMITTED (SQL Server).

Q3. What is a Deadlock?

Two or more transactions waiting on each other in a cycle.

Q4. What is Blocking?

One transaction waits for another to release a lock.

Q5. What is Parameter Sniffing?

Execution plan reuse based on initial parameter values.

Q6. What is Partitioning?

Splitting large tables into manageable partitions.

Q7. Why use Execution Plans?

To understand how SQL Server executes queries and identify bottlenecks.

Q8. Why avoid SELECT *?

It increases I/O and may prevent optimal index usage.

Q9. What is Snapshot Isolation?

An isolation level that uses row versioning to provide consistent reads with reduced blocking.

Q10. Clustered Index?

Defines the physical order of table data.

Q11. Non-Clustered Index?

A separate index structure containing keys and row locators.

Q12. Covering Index?

An index that contains all data required by a query.

Q13. Temp Table vs Table Variable?

Temporary tables generally perform better for larger datasets.

Q14. Why use Stored Procedures?

Reusable database logic with parameterization and centralized management.

Q15. SQL Server or PostgreSQL?

Choose based on business, technical, operational, and licensing requirements.

Module 7 Completed

You now have comprehensive senior-level coverage of:

- Indexes
- Covering Indexes
- Execution Plans
- Query Optimization
- Stored Procedures
- Views
- CTEs
- Window Functions
- Transactions
- ACID
- Isolation Levels
- Deadlocks
- Locking & Blocking
- Parameter Sniffing
- Partitioning
- SQL Server vs PostgreSQL
- JSON Support
- Production Troubleshooting
- Rapid-Fire Interview Questions

Next Recommended Module: Microservices Architecture (Most Asked for Senior Roles)

This module will cover approximately **50+ senior interview questions**, including:

- Monolith vs Microservices
- Service-to-Service Communication

- API Gateway
- CQRS
- Event-Driven Architecture
- Saga Pattern
- Outbox Pattern
- Distributed Transactions
- Circuit Breaker
- Retry Policies (Polly)
- Service Discovery
- Distributed Caching
- Distributed Logging & Tracing
- Azure Service Bus Integration
- Kubernetes Considerations
- Real Production Architecture Scenarios

This is one of the most important modules for senior .NET developers because architecture and distributed systems are frequently discussed in technical interviews.